

Polymarket Mini: A Smart-Contract Prediction Market with Signed CLOB Orders

Project type: Implementation **Repository:** <https://github.com/kapelame/polymarket-mini> **Main components:** Solidity smart contracts, EIP-712 signed orders, local CLOB server, optimistic oracle, Next.js trading UI

Abstract

Polymarket Mini is an educational implementation of a Polymarket-style binary prediction market. The project demonstrates how a prediction market can be built from three cryptographic and smart-contract ideas: conditional tokens, signed off-chain orders, and oracle-based settlement. Users can create a binary market, trade YES/NO outcome shares through a central limit order book, and redeem winning tokens after resolution. Smart contracts handle collateral custody, conditional token minting, order verification, atomic settlement, and final payout. The off-chain server handles latency-sensitive exchange functions: order book management, matching, account history, live market data, and demo automation.

The goal is not to reproduce Polymarket's production infrastructure exactly, but to implement its core protocol shape in a compact codebase. The demo supports a full one-minute lifecycle: create a BTC price market, seed liquidity, place signed orders, match trades, show portfolio/P&L, resolve the market, and redeem the winning side.

Motivation

Prediction markets turn beliefs about future events into tradable prices. In a binary market, one YES token and one NO token together represent one unit of collateral. If the event happens, YES redeems for one dollar and NO redeems for zero; if the event does not happen, the opposite occurs. Before resolution, the price of YES can be interpreted as the market's probability estimate for the event.

This project naturally combines several topics from cryptography and blockchains: commitments to market identities, token IDs derived from hashed conditions, authenticated user intent through digital signatures, on-chain settlement rules, and an oracle protocol for turning a real-world event into a final blockchain payout.

System Overview

The system has four layers.

First, the **conditional token layer** (`src/core/ConditionalTokens.sol`) implements a simplified version of the Gnosis Conditional Tokens Framework. It locks USDC collateral and mints ERC-1155 outcome tokens. A binary market has two index sets: YES = 1 and NO = 2. Depositing one unit of USDC can mint one YES token and one NO token. Holding both sides is equivalent to holding the original collateral, so users can merge a complete set back into USDC before resolution.

Second, the **exchange layer** (`src/exchange/CTFExchange.sol`) verifies EIP-712 orders and settles matched trades. Orders are created off-chain and signed by the maker. The contract recomputes the EIP-712 digest, recovers the signer, checks remaining fill amount, and performs an atomic transfer of USDC and conditional tokens. The exchange supports direct BUY/SELL matching, complementary BUY matching that mints YES/NO from collateral, and complementary SELL matching that merges YES/NO back into collateral.

Third, the **oracle layer** (`src/oracle/OptimisticOracle.sol`) resolves markets. It is modeled after optimistic oracle designs: after expiration, a proposer posts a bonded YES or NO answer; anyone may dispute during a one-hour dispute window; undisputed answers settle automatically, while disputed answers go to a trusted arbitrator. Settlement calls `reportPayouts` with `[1, 0]` for YES or `[0, 1]` for NO.

Fourth, the **application layer** contains an Express CLOB server and a Next.js UI. The server stores local books, verifies signatures, submits settlement transactions as the operator, tracks fills and open orders, and exposes portfolio data. The UI provides market pages, dynamic BTC candles, order book, buy/sell controls, trade history, account positions, P&L, and one-click local demo flows.

Cryptographic Protocol Design

The most important cryptographic object is the conditional position ID. A market starts with a human-readable question, which is hashed into a `questionId` . The condition ID is derived as:

```
conditionId = keccak256(oracle, questionId, outcomeSlotCount)
```

For each outcome, the collection ID commits to the parent collection, condition, and selected outcome bitmask:

```
collectionId = keccak256(parentCollectionId, conditionId, indexSet)
```

Finally, the ERC-1155 token ID is derived from the collateral token and collection:

```
positionId = keccak256(collateralToken, collectionId)
```

This derivation means the identity of a YES or NO token is deterministically bound to the oracle, question, number of outcomes, collateral asset, and outcome set. Changing any part of the condition changes the final token ID.

The second cryptographic object is the signed exchange order. The order contains the maker, signer, token ID, amounts, side, expiration, nonce, fee rate, and signature type. The EIP-712 domain binds the signature to the local chain ID and exchange contract address:

```
EIP712Domain(  
  name = "CTFExchange",  
  version = "1",  
  chainId,  
  verifyingContract  
)
```

The order hash follows the standard EIP-712 pattern:

```
orderHash = keccak256("\x19\x01", domainSeparator, structHash)
```

The contract uses `ECDsa.recover` to require that the recovered signer equals the maker. This prevents the CLOB operator from changing prices, token IDs, sides, amounts, or expirations after a user signs. The operator can choose which valid orders to match, but it cannot fabricate a user's order or settle an altered one.

The CLOB server also implements account authentication. L1 authentication proves wallet ownership with an EIP-712 `ClobAuth` message. L2 API calls use HMAC signatures over timestamp, HTTP method, path, and body. This mirrors common trading-system design: a wallet signature establishes account control, and cheaper API-key signatures authenticate frequent requests.

Market Lifecycle

A complete market flow works as follows.

1. The market creator defines a binary question and expiration time. For the demo, the app creates a BTC market with a short horizon.
2. `ConditionalTokens.prepareCondition` registers the condition with the oracle and fixes the number of outcome slots.
3. The exchange registers the YES and NO position IDs so it can validate complementary orders.
4. Liquidity is created by splitting USDC into YES and NO tokens, or by matching complementary buy orders that collectively fund a full collateral unit.
5. A user signs a BUY or SELL order using EIP-712 typed data. The order is submitted to the CLOB server.
6. The server verifies the signature off-chain, places the order in the local book, and attempts to match it.
7. When a match is found, the server calls the exchange contract as the operator. The contract verifies both signatures and atomically transfers USDC and ERC-1155 outcome tokens.
8. After expiration, the oracle proposes an answer, waits through the dispute flow, and reports payouts to the conditional token contract.
9. Winning token holders call `redeemPositions` to burn their outcome tokens and receive USDC collateral.

This creates a hybrid structure: the order book is fast because it is off-chain, but custody and settlement stay on-chain. Users do not trust the server with collateral, and invalid matches fail at the smart-contract layer.

Implementation Details

The smart contracts are written in Solidity 0.8.20 and tested with Foundry. `ConditionalTokens.sol` implements prepare, split, merge, resolve, and redeem operations. `CTFExchange.sol` implements order hashing, signature verification, registration, cancellation, direct matching, complementary mint settlement, and complementary merge settlement. `OptimisticOracle.sol` implements market preparation, answer proposal, dispute, undisputed settlement, and arbitrated dispute resolution. `MockUSDC.sol` provides local collateral.

The server is written in Node.js/Express. It maintains local books using `clob-server/src/orderbook/OrderBook.js`, verifies order signatures using the same hashing rules as the Solidity exchange, and submits settlement transactions through `clob-server/src/chain/Settlement.js`. Recent trades, open orders, and account data are stored locally so the UI can show a complete demo portfolio. The market factory builds one-minute BTC markets for presentations and can seed initial liquidity so a user can experience the full flow quickly.

The frontend is written in Next.js. It exposes market creation, trading, order book visualization, dynamic candlestick data, wallet/demo account state, historical fills, positions, and total P&L. The UI follows the Polymarket mental model: markets are questions, YES and NO are tradable assets, and the account panel shows how orders become positions and then redeemable value after settlement.

Evaluation

The implementation was evaluated with Foundry unit tests and gas reporting. The current test suite contains 32 tests across five suites:

Test suite	Scope	Result
<code>ConditionalTokensTest</code>	condition preparation, splitting, merging, payout reporting, redemption, revert cases	9 passed
<code>OptimisticOracleTest</code>	proposal, dispute, undisputed settlement, arbitrator resolution, redemption	10 passed
<code>CTFExchangeTest</code>	direct matching, cancellation, invalid signatures, expired orders, operator check	5 passed
<code>CTFExchangeMintTest</code>	complementary BUY matching and mint settlement	4 passed
<code>CTFExchangeMergeTest</code>	complementary SELL matching and merge settlement	4 passed

The command `forge test --gas-report` completed with **32 passed, 0 failed, 0 skipped**. Selected gas measurements are:

Operation	Average gas
ConditionalTokens.prepareCondition	51,137
ConditionalTokens.splitPosition	109,450
ConditionalTokens.mergePositions	50,995
ConditionalTokens.redeemPositions	40,422
CTFExchange.registerOrder	52,891
CTFExchange.matchOrders	61,861
CTFExchange.matchComplementaryOrders	155,480
CTFExchange.matchComplementarySellOrders	124,184
OptimisticOracle.prepareMarket	110,702
OptimisticOracle.proposeAnswer	110,037
OptimisticOracle.settle	99,609

Deployment costs were approximately 1.80M gas for ConditionalTokens , 1.91M gas for CTFExchange , and 1.08M gas for OptimisticOracle . The most expensive exchange path is complementary order matching because it combines order verification, collateral transfer, conditional token splitting, and ERC-1155 transfers in one atomic transaction. Direct matching is cheaper because it only swaps existing assets.

The tests also evaluate security-relevant failure cases: duplicate condition preparation, insufficient approvals, redemption before resolution, wrong oracle reports, proposing before expiry, early settlement, invalid order signatures, expired orders, non-operator settlement, same-token complementary matching, unregistered token matching, and merge amount mismatches.

Security Discussion and Limitations

The system protects users against several important failures. Conditional token IDs are deterministic and collision-resistant under keccak256 . Orders are bound to the chain and exchange address through EIP-712, so signatures cannot be replayed on a different deployment. Settlement is atomic: either both sides transfer correctly, or the transaction reverts. Payouts can only be reported by the committed oracle address.

However, this is still a local prototype. The CLOB operator is trusted for liveness and ordering. It cannot create fake signed orders, but it can censor orders or choose match priority. The optimistic oracle uses a trusted arbitrator and demo-oriented timing assumptions; production would require a stronger data source, decentralized dispute process, or integration with an established oracle protocol. Fee accounting, global nonce invalidation, proxy wallet signatures, production persistence, and MEV resistance are future work.

The demo uses local Anvil accounts and mock USDC, so it should be evaluated as a protocol implementation and teaching artifact. Its value is that each important step is inspectable: the user signs an order, the server verifies it, the exchange contract verifies it again, settlement moves collateral and outcome tokens, and the oracle eventually turns a real-world answer into redeemable collateral.

Conclusion

Polymarket Mini implements the core mechanics of a cryptographic prediction market in a compact repo. The project combines conditional token construction, EIP-712 signed orders, on-chain exchange settlement, and optimistic oracle resolution into a working end-to-end demo. The implementation demonstrates why prediction markets are a good fit for smart contracts: custody, settlement, and redemption are rule-based and auditable, while market making and order matching can remain off-chain for speed.

The final result is a usable local application and a tested smart-contract protocol. A user can create a BTC market, place trades, view order history and P&L, resolve the outcome, and redeem winnings. The accompanying tests and gas report show that the protocol's main flows and revert cases are covered, satisfying the implementation-and-evaluation goals of the project.

Reproduction Commands

```
npm run setup
anvil
PRIVATE_KEY=<anvil-account-private-key> \
OPERATOR_ADDRESS=<operator-wallet-address> \
forge script script/Deploy.s.sol:Deploy --rpc-url http://localhost:8545 --broadcast
cp clob-server/.env.example clob-server/.env
npm run dev
```

For tests and evaluation:

```
forge test --gas-report
npm run check
```